

Ethics as a Service in Service-oriented Software Engineering

Part 6 – Let's build a useful EaaS block

Embedded Special Topic for Service-oriented Software Engineering

University of L'Aquila

Patrizio Migliarini, PhD

Let's build a useful EaaS block

What we are building

We are building a REST Ethics as a Service.

The input is a pull request that changes authentication logic in a healthcare system.

The service must not trust a pre-declared risk value, because the caller may underestimate or misclassify the risk.

The service computes risk from the actual code change through a simulated LLM review, then applies explicit policies and stores an audit trail.

The EaaS block is not a blind oracle. It is a pipeline. Every stage leaves evidence.

Let's build a useful EaaS block

Metadata

`/example-pr/metadata.json`

It contains:

request ID, caller, system name, repository, domain, jurisdiction, task type, affected stakeholders, policy bindings, evidence artefacts, and changed file descriptors.

It NOT contains:

risk_label.

There is no decision. The caller is not telling EaaS what the risk is. The caller is only providing context and selecting the policies that apply to the request.

Let's build a useful EaaS block

Old and New code

```
/example-pr/old/AuthService.java  
/example-pr/new/AuthService.java
```

In the old file, point out rate limiting, passwordHasher.verify, session rotation, and audit logging.

In the new file, point out the comments marked EaaS-DEMO CHANGE.

EaaS is not reviewing an abstract ethical dilemma.

It receives code.

The LLM simulation will review these exact changes.

Let's build a useful EaaS block

Policies

```
/policies/auditability_required.json  
/policies/privacy_healthcare.json  
/policies/provenance_required.json  
/policies/security_sensitive_change.json
```

Each policy contains an ID, version, owner, description, hard-blocking findings, escalation findings, required evidence, and a decision rule.

The policy is not hidden inside the model or inside the server.
It is an external artefact that can be reviewed, versioned, and changed.

Let's build a useful EaaS block

Server

```
/python-eaas/server.py  
    audit_store.py  
    eaas_pipeline.py  
    policy_repository.py
```

```
/java-eaas/EaasJavaServer.java  
    AuditStore.java  
    EaasPipeline.java  
    PolicyRepository.java  
    Json.java (we need a JSON parser)
```

The two services expose the same contract.

Python runs on port 8090, Java runs on port 8091.

EaaS contract and pipeline can be implemented independently in different technologies.

Let's build a useful EaaS block

Request builder

```
/tools/build_request.py  
/tools-java/src/eaas/tools/BuildRequest.java
```

The JSON sent to the service is not hand-written.

It is assembled from metadata plus the old and new code files.

This keeps the example readable: we can show metadata separately, code separately, and still send a complete request to the service.



Let's build a useful EaaS block

LLM simulation

```
/tools/build_and_call.py --port 8090 --endpoint llm  
  
(/tools-java/src/eaas/tools/BuildAndCall.java)
```

The LLM interaction is not invisible.

The prompt is versioned, it has a hash.

The raw response is stored.

The parsed response is visible.

The risk is computed here, not declared by the caller.

Let's build a useful EaaS block

LLM simulation output

```
SEC_AUTH_BYPASS  
SEC_PASSWORD_VERIFICATION_WEAKENED  
SEC_RATE_LIMIT_REMOVED  
SEC_SESSION_ROTATION_REMOVED  
AUD_AUDIT_LOGGING_DISABLED  
PRI_PATIENT_DATA_LOGGED
```

The simulated LLM is not giving us a vague paragraph.

It returns structured findings.

That is what allows the next stage, the policy engine, to behave deterministically and auditably.

Let's build a useful EaaS block

Full assessment

```
/tools/build_and_call.py --port 8090 --endpoint assess
```

```
(/tools-java/src/eaas/tools/BuildAndCall.java)
```

```
decision: REJECT  
llm_computed_risk_level: critical  
llm_computed_risk_score: 64  
hard_policy_blocks  
required_actions  
audit_uri
```

This is the difference between LLM analysis and EaaS governance.

The LLM computes risk and findings.

The EaaS policy engine **maps those findings to organizational policy**.

The rejection is not only because the model thinks the PR is bad.

It is because bound policies classify some findings as hard blockers.

thank you